

PULSE Instrument Cluster

System Requirements Specification

Ahmed Abdelghany

2026-09-23

Contents

Document control	1
Change log	1
1. Introduction	2
1.1 Purpose	2
1.2 Conventions	2
1.3 System overview	3
2. Functional requirements	3
2.1 Vehicle data abstraction	3
2.2 Telemetry source	4
2.3 Human-machine interface	5
2.4 Comparison and evidence	6
2.5 Driver monitoring and closed loop	6
2.6 Autonomy provider	7
3. Interface requirements	7
4. Non-functional requirements	8
5. Constraints and assumptions	9
6. Requirement summary at week 3	9

Document control

Field	Value
Document ID	PULSE-SyRS-001
Title	System Requirements Specification
Version	1.5
Status	Draft frozen for review
Date	2026-10-02
Author	Ahmed Abdelghany
Reviewer	Cockpit Electronics / HMI Platform Group (sponsor)
Review gate	Sprint 1 review, week 3, 4 September 2026
Parent	PULSE-StRS-001
Related	PULSE-SAD-001, PULSE-SAF-001, PULSE-SEC-001, PULSE-RTM-001

Change log

Version	Date	Author	Change
0.1	2026-08-25	A. Abdelghany	Requirements FR-1 to FR-17, IR-1 to IR-5, NFR-1 to NFR-8 drafted
1.0	2026-09-03	A. Abdelghany	Verification method, week-3 status and evidence added per requirement; driver monitoring (FR-18 to FR-21) and autonomy provider (FR-22) added; safety and security requirements moved to PULSE-SAF-001 and PULSE-SEC-001; frozen for review
1.1	2026-09-03	A. Abdelghany	Requirement-group map added to the system overview
1.2	2026-09-03	A. Abdelghany	Evidence for FR-3, FR-4, FR-6 and FR-11 now points at the regression suite (scripts/regression.sh) and its committed reports
1.3	2026-09-08	A. Abdelghany	FR-13 corrected to Partial: the two HMIs have diverged to 28 and 17 subscribed signals. NFR-8 and IR-2 evidence updated for the pinned broker image and the new unit-sanity check. FR-19 and FR-20 reworded, FR-20 now citing PULSE-SAF-001 SM-3 instead of restating it. Section 6 count table replaced by a pointer to the generated matrix. Worked requirement row added to section 1.2
1.4	2026-09-23	A. Abdelghany	Sprint 2 status: IR-5 and NFR-8 Implemented (secure-by-default broker, Flutter SDK pinned), FR-3 regeneration check now executes, FR-8 proven on a virtual bus, FR-13 wording updated, NFR-5 reduced to Partial pending a host with AVX
1.5	2026-10-02	A. Abdelghany	FR-13 to Implemented: the Compose cluster now consumes the same 28 signals. FR-8 evidence extended to the simulator's CAN mode and the can-decode and can-path checks. FR-12 and NFR-5 evidence updated from a second host

1. Introduction

1.1 Purpose

This document states what the PULSE system shall do, in numbered, verifiable requirements. It realises the stakeholder needs in PULSE-StRS-001. The architecture that satisfies these requirements is in PULSE-SAD-001. Functional-safety and cybersecurity requirements are held in PULSE-SAF-001 and PULSE-SEC-001 so that their respective reviewers own them.

1.2 Conventions

Requirement IDs are stable. A withdrawn requirement keeps its number and is marked withdrawn; a new one takes the next free number. "Shall" is binding. "Should" is a recommendation.

Verification method uses the standard four:

Code	Method	Meaning
I	Inspection	Reading code, configuration or documents
A	Analysis	Reasoning, calculation or static analysis
D	Demonstration	Running the system and observing behaviour, no measured pass criterion
T	Test	Executed check with a recorded pass or fail

Status at week 3 uses:

Status	Meaning
Implemented	Met by the current build; evidence named
Partial	Partly met; the gap and its sprint are named
Planned S2 / S3 / S4	Not yet started; scheduled for that sprint

Read together, one row looks like this. FR-11 says the HMI shall subscribe to abstract signals only. Its verification code is T, so a check either passes or fails rather than being read or argued. Its status is Implemented. Its evidence names the check that proves it, hmi-purity, which scans both HMI sources for CAN, DBC and SocketCAN tokens and fails the build if it finds one. Every requirement row in this document is read the same way: what shall happen, how it is proved, where it stands, and what to open to see for yourself.

1.3 System overview

PULSE is an instrument-cluster demonstrator on a standards-based software-defined-vehicle data stack. A KUKSA databroker serves a COVESA VSS 6.0 signal tree with overlays. Providers (a drive-cycle simulator, a camera-based driver monitor, later a CAN feeder and an Autoware bridge) publish into it. Two HMIs built from one visual design, Flutter on Linux and Jetpack Compose on Android Automotive, subscribe from it. Full detail is in PULSE-SAD-001.

Where each requirement group applies

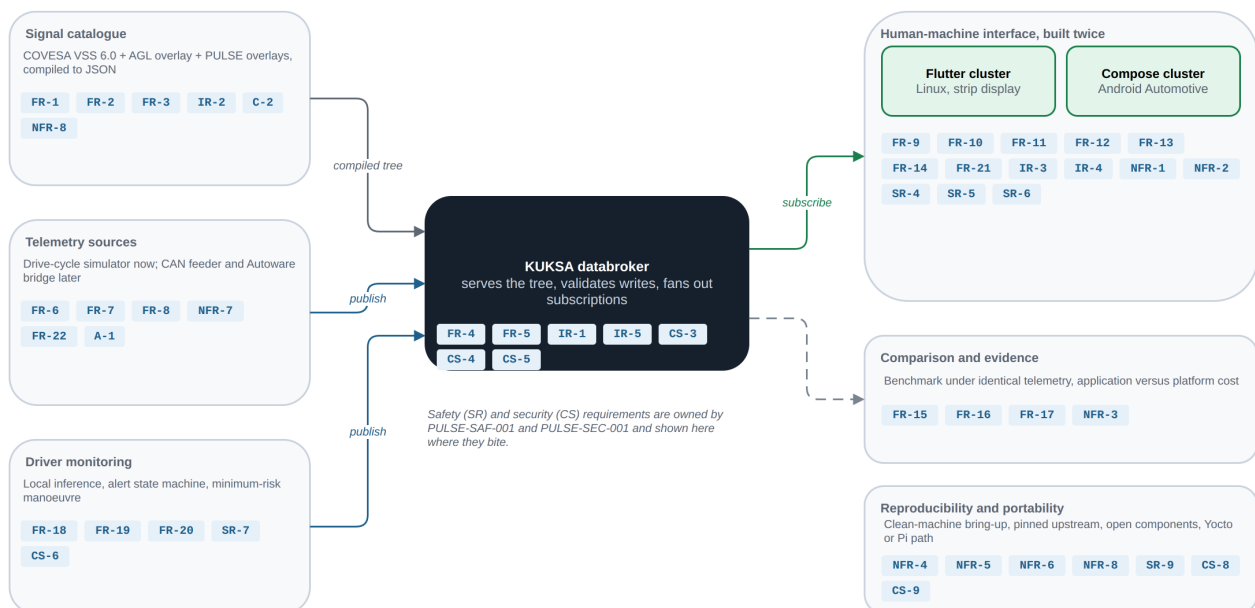


Figure 1: Where each requirement group applies on the system. Safety and security IDs are shown where they bite; their text lives in PULSE-SAF-001 and PULSE-SEC-001.

Source: docs/diagrams/requirements-map.drawio.

2. Functional requirements

2.1 Vehicle data abstraction

ID	Requirement	Verification	Status (W3)	Evidence
FR-1	The system shall represent all vehicle data using an open, standardised signal catalogue (COVESA VSS), not proprietary or bus-specific identifiers.	I	Implemented	vss/vss_agl.json compiled from VSS 6.0; 1,302 leaf signals
FR-2	The system shall extend the standard catalogue for domain-specific signals using the standard overlay mechanism only, with no forks of the upstream specification.	I	Implemented	vss/agl_vss_overlay.vspec, vss/pulse_vss_overlay.vspec, vss/dms_vss_overlay.vspec; upstream spec consumed unmodified
FR-3	The compiled signal tree shall be reproducible from versioned source files by a documented command.	T	Implemented	Suite check vss-regeneration, byte-identical with vss-tools 6.0 in .venv-tools; first executed 2026-09-23, it had skipped before for want of the tool
FR-4	A central broker shall serve the signal tree and allow multiple independent clients to subscribe concurrently.	D	Implemented	KUKSA databroker 0.7.0; Flutter cluster, Compose cluster and engine-audio service subscribe concurrently; suite check broker-up
FR-5	Signal producers and consumers shall be mutually decoupled: neither needs knowledge of the other's implementation.	I	Implemented	Providers and HMIs share no code, only VSS paths (PULSE-SAD-001 section 4)

2.2 Telemetry source

ID	Requirement	Verification	Status (W3)	Evidence
FR-6	The system shall provide a telemetry source that generates physically plausible, correlated vehicle behaviour (speed, engine speed, gear, temperature, fuel, odometer) without any vehicle hardware.	D	Implemented	emulator/telemetry_sim.py: lap and cruise cycles with correlated RPM, fuel and odometer. Suite check live-telemetry passes
FR-7	The telemetry source shall run a repeatable cycle so that UI behaviour can be compared across runs and across implementations.	T	Partial	Phase table is fixed with no randomness, but sampling is wall-clock driven, so runs differ in timing. Fixed-step replay in S4

ID	Requirement	VerificationStatus (W3)		Evidence
FR-8	The telemetry source shall be replaceable by a real vehicle-bus provider without modification to any HMI code.	T	Partial	Proven on a virtual bus on 23 September (scripts/can_spike.py). Since 2 October the simulator can act as the vehicle's ECUs (SIM_CAN=vcan0, CAN=1 ./run.sh): the 35 signals the catalogue maps from AGL's DBC leave it only as frames and reach the broker through KUKSA's CAN provider (emulator/can_bus.py). Suite check can-decode verifies all 35 at minimum, midpoint and maximum and runs in CI; can-path runs the live path wherever vcan0 is up. Physical bus in S3

2.3 Human-machine interface

ID	Requirement	VerificationStatus (W3)		Evidence
FR-9	The HMI shall display, updating live: road speed, engine speed with warning and redline indication, selected gear, fuel level and range, per-corner tyre pressures, and driver-relevant status indicators.	D	Implemented	pulse-cluster/lib/screen/pulse_screen.dart; subscribed paths listed in PULSE-SAD-001 appendix A
FR-10	The HMI shall display motorsport telemetry: current lap number, current and best lap time, aerodynamic (DRS) state, and energy-recovery (ERS) level and state.	D	Implemented	Vehicle.Motorsport.* overlay signals rendered by both HMIs
FR-11	The HMI shall subscribe to abstract signals only. It shall not contain CAN frame parsing, DBC knowledge, or hardware-specific decoding.	T	Implemented	Regression suite check hmi-purity scans both HMI sources for CAN, DBC and SocketCAN tokens; passing, report in docs/evidence/
FR-12	The HMI shall be implemented twice, once in Flutter for Linux and once in Compose for Android Automotive, from a single shared visual design.	I	Implemented	pulse-cluster/ (Flutter) and pulse-cluster-android/ (Compose); design source in design/. Both seen running on 2 October, Compose on the Android 16 car emulator (suite check compose-ui, screens under docs/evidence/screens/)

ID	Requirement	VerificationStatus (W3)		Evidence
FR-13	Both implementations shall consume the identical signal source, so any observed difference is attributable to the platform, not the data.	D	Implemented	Since 2 October both clusters subscribe to the same 28 paths through the same broker, protocol, catalogue limits and freshness rules. The benchmark figures in BRIEF.md were measured before parity and are re-measured in S4 (FR-15)
FR-14	The HMI shall render full-screen on the target strip display, and shall degrade gracefully to a normal window when that display is absent.	D	Implemented	run.sh locates a 2560 x 720 output at runtime and passes CLUSTER_GEOM; unset means a normal window

2.4 Comparison and evidence

ID	Requirement	VerificationStatus (W3)		Evidence
FR-15	The system shall support measurement of both implementations under identical telemetry: CPU utilisation, memory footprint, and shipped artefact size.	T	Partial	Measured once by hand: 123 vs 46 MB memory, 12.2 vs 11.5 % CPU, 50 vs 23 MB artefact (BRIEF.md). Scripted benchmark in S4
FR-16	Measurements shall distinguish application-level cost from platform-level cost, since these drive different procurement decisions.	A	Partial	Platform memory reported separately (AAOS 3.2 to 4.0 GB from emulator); target-hardware figures planned S3 to S4
FR-17	Results shall be documented such that a reviewer can reproduce them from the repository.	T	Planned S4	Benchmark script and results file to be committed under docs/

2.5 Driver monitoring and closed loop

ID	Requirement	VerificationStatus (W3)		Evidence
FR-18	The system shall derive driver fatigue and distraction levels from a driver-facing camera using inference executed entirely on the local device, and publish them as VSS driver signals.	D	Implemented	emulator/dms.py: MediaPipe FaceLandmarker on CPU; publishes Vehicle.Driver.FatigueLevel, DistractionLevel, AttentiveProbability, IsEyesOnRoad at 10 Hz
FR-19	The system shall maintain a driver alert state of NONE, DROWSY, DISTRACTED or NO_FACE. It shall raise an alert only after a configurable hold time, and release it only after a configurable clear time, so a momentary event does not flicker the display.	T	Implemented	DMS_ALERT_HOLD_S 3.0 s, DMS_ALERT_CLEAR_S 2.0 s; published as Vehicle.Driver.Monitoring.AlertState

ID	Requirement	VerificationStatus (W3)		Evidence
FR-20	When cruise mode is active and the alert state is DROWSY, the vehicle model shall execute the minimum-risk manoeuvre specified in PULSE-SAF-001 SM-3, which the driver can abort by recovering attention during the countdown.	T	Implemented	telemetry_sim.py cruise mode implements SM-3; published as Vehicle.Driver.Monitoring.InterventionState and InterventionCountdown
FR-21	The HMI shall render the driver attention level, the active alert as a full-width banner distinguishable by colour per alert type, and the intervention countdown.	D	Implemented	Attention panel and banner in pulse_screen.dart; attention meter in classic_screen.dart

2.6 Autonomy provider

ID	Requirement	VerificationStatus (W3)		Evidence
FR-22	The system shall accept an autonomy-stack provider that publishes planned trajectory, detected objects and drive mode into the broker as VSS signals, so that the cluster can render an autonomy state without knowing the stack behind it.	D	Planned S3	Autoware planning-simulation bridge, work package 04; VSS overlay for autonomy signals to be added under vss/

3. Interface requirements

ID	Requirement	VerificationStatus (W3)		Evidence
IR-1	Client-to-broker communication shall use a language-neutral RPC interface supporting streaming subscriptions.	I	Implemented	gRPC over HTTP/2, kuksa.val.v1 Subscribe stream; Dart and Kotlin stubs generated from the same .proto
IR-2	Signal semantics (units, ranges, allowed enumeration values, and encodings) shall be defined in the signal catalogue, not in client code.	I	Implemented	Units, allowed lists and max in the .vspec overlays; broker rejects out-of-catalogue values on set; suite check unit-sanity catches a value published in the wrong unit
IR-3	The target display interface shall be a 2560 x 720 automotive strip panel.	D	Implemented	Corsair Xeneon Edge on the desk rig; layout designed at 2560 x 720
IR-4	Display placement shall be configurable at runtime, not fixed at compilation.	D	Implemented	CLUSTER_GEOM environment variable read by the Linux runner; run.sh derives it from xrandr

ID	Requirement	VerificationStatus (W3)		Evidence
IR-5	The system shall run without transport security in the local development configuration, and the security posture shall be an explicit, documented choice.	I	Implemented	Secure by default. Plaintext only with PULSE_INSECURE=1, which prints a boxed warning at start-up; suite check transport-security skips and says so

4. Non-functional requirements

ID	Requirement	VerificationStatus (W3)		Evidence
NFR-1	Responsiveness: displayed values shall track the telemetry source with no perceptible lag at a glance.	D	Implemented	Fast signals at 10 Hz, one broker tick from publish to render; observed at the review demo
NFR-2	Legibility: primary values (speed, gear, engine speed) shall be readable in a sub-second glance, consistent with driver-distraction practice.	D	Implemented	Design reviewed on the strip; numerals sized for the 720 px height
NFR-3	Footprint: the Linux implementation shall be deployable on constrained embedded hardware; total platform footprint is a first-class evaluation criterion.	T	Planned S3	Raspberry Pi 5 baseline, work package 03, week 9
NFR-4	Portability: migration to a different SoC or to a Yocto/AGL image shall not require HMI rework.	A	Planned S3	Argument to be recorded with the Pi 5 baseline; Flutter embedder is the only platform-specific layer
NFR-5	Reproducibility: a new engineer shall be able to bring up the full stack on a clean machine from documentation alone.	T	Partial	Bring-up of 2026-08-18 in ONBOARDING.md. On 2026-10-02 a second host rebuilt both environments from requirements.txt with constraints.txt and ran every provider, the driver monitor included; a fresh install matches the SBOM exactly. Re-run on the Pi 5 in S3
NFR-6	Openness: the stack shall be built from open-source components, with no proprietary runtime licence required for evaluation.	I	Implemented	KUKSA (Apache-2.0), VSS (MPL-2.0), Flutter (BSD), MediaPipe (Apache-2.0), AOSP; NOTICE file for the face model
NFR-7	Determinism: repeated runs of the same cycle shall produce the same signal sequence, so comparisons are valid.	T	Partial	Same as FR-7: value sequence is deterministic in shape, sample timing is not. Fixed-step replay planned S4

ID	Requirement	VerificationStatus (W3)		Evidence
NFR-8	Maintainability: upstream components shall be consumed at pinned versions, not floating heads.	I	Implemented	Broker by tag and digest; Python by requirements.txt on Python 3.11; VSS 6.0 and vss-tools 6.0; Dart by pubspec.lock; Flutter 3.47.0 in .fvmrc, enforced by suite check toolchain-pin

5. Constraints and assumptions

Constraints C-1 to C-5 and assumptions A-1 to A-3 are owned by PULSE-StRS-001 section 5 and apply here unchanged.

6. Requirement summary at week 3

The counts are not repeated here. PULSE-RTM-001 section 1 computes them from the requirement rows in this document every time the documents are built, so it is always current and this page could only drift away from it.

That matrix also covers safety (SR) and security (CS) requirements, which live in PULSE-SAF-001 and PULSE-SEC-001.